

1. Clustered vs Non-Clustered Indexes

An index is like the index at the back of a textbook. Instead of reading every page to find "HashMap", you look up "HashMap" in the index, find "page 142", and go directly there. Without an index, the database does a **full table scan** — reading every single row.

A clustered index **determines the physical order of data on disk**. The table data IS the index.

- There can be **only ONE** clustered index per table (data can only be physically sorted one way)
- In most databases, the **Primary Key** automatically creates a clustered index
- The leaf nodes of a clustered index contain the **actual data rows**

Clustered Index on employee_id: [50] / \ [20,30] [60,80] / \ / \ [10,20][30,40][50,60][70,80,90] ← These ARE the actual data rows ↑ Data is physically sorted by employee_id on disk

Real World Analogy: A dictionary. Words are physically arranged in alphabetical order. The arrangement IS the index. You can only have one physical ordering (alphabetical).

A non-clustered index is a **separate structure** that points back to the actual data. Like a book's back-of-book index.

- You can have **multiple** non-clustered indexes per table
- Leaf nodes contain the **indexed column value + a pointer** (row locator) to the actual data
- Requires an extra "lookup" step to fetch the full row

Non-Clustered Index on last_name: [Kumar] / \ [Gupta,Jain] [Patel,Singh] / \ / \ [Agarwal] [Gupta] [Kumar]
[Patel,Singh] ↓ ↓ ↓ ↓ ptr→Row5 ptr→Row2 ptr→Row1 ptr→Row3,Row7 ↑ Leaf nodes have pointers, NOT actual data

Real World Analogy: The index at the back of a textbook. It says "HashMap — page 142". The index is separate from the actual content. You can have multiple indexes (one by topic, one by author name, etc.).

Visual Comparison

Clustered Non-Clustered  Structure: B+ Tree where leaves ARE the data rows have pointers to data Per table: Only 1 Many (up to ~999) Speed (exact): Faster (data is right Slower (extra lookup there at the leaf) to fetch actual row) Speed (range): Excellent (data is Slower (pointers may physically contiguous) point to scattered rows) Example: PRIMARY KEY INDEX on email, phone, etc.

When Data is Fetched

Clustered Index Lookup:

```
Query: SELECT * FROM employees WHERE employee_id = 42; Step 1: Traverse B+ Tree → Find leaf node with id=42  
Step 2: Leaf node HAS the full row → Done! Total: 1 lookup
```

Non-Clustered Index Lookup:

```
Query: SELECT * FROM employees WHERE last_name = 'Kumar'; Step 1: Traverse Non-Clustered B+ Tree → Find leaf  
with 'Kumar' Step 2: Leaf has pointer (e.g., Row ID 7) Step 3: Go to the actual table (or clustered index) →  
Fetch Row 7 Total: 2 lookups (this extra step is called a "bookmark lookup" or "key lookup")
```

Covering Index (Pro Tip)

If your non-clustered index **includes all the columns** the query needs, the DB doesn't need the second lookup:

```
-- Query: SELECT email FROM employees WHERE last_name = 'Kumar'; -- Create a covering index: CREATE INDEX  
idx_lastname_email ON employees(last_name) INCLUDE (email); -- Now the index leaf has both last_name AND  
email → no bookmark lookup needed!
```

2. Why a Query Can Be Slow Even WITH an Index

This is a very practical interview question. Here are the real reasons:

Reason 1: The Query Doesn't USE the Index (Index Not Picked)

The optimizer may skip your index if:

a) Function on indexed column:

```
-- BAD: Index on created_date is useless here SELECT * FROM orders WHERE YEAR(created_date) = 2025; -- GOOD:  
Rewrite as range scan SELECT * FROM orders WHERE created_date >= '2025-01-01' AND created_date <  
'2026-01-01';
```

b) Implicit type conversion:

```
-- Column phone_number is VARCHAR, but you pass a number SELECT * FROM users WHERE phone_number = 9876543210;  
-- DB converts every row's phone_number to number → full scan! -- GOOD: SELECT * FROM users WHERE  
phone_number = '9876543210';
```

c) Leading wildcard in LIKE:

```
-- BAD: Can't use index (has to check every row) SELECT * FROM products WHERE name LIKE '%phone%'; -- OK: Can  
use index (searches from the start) SELECT * FROM products WHERE name LIKE 'phone%';
```

d) OR conditions mixing indexed and non-indexed columns:

```
-- Index exists on email but NOT on city SELECT * FROM users WHERE email = 'test@test.com' OR city = 'Mumbai'; -- DB may do a full scan because it can't efficiently combine the results
```

Reason 2: Low Selectivity (Index Returns Too Many Rows)

```
-- Column 'gender' has only 2 values: M and F -- An index on gender returns ~50% of the table SELECT * FROM employees WHERE gender = 'M'; -- The optimizer thinks: "I'd have to do 500K bookmark lookups on a 1M row table. -- A full table scan reading sequentially is actually faster." -- → Index IGNORED.
```

Rule of thumb: If an index would return more than ~15-20% of the table, the optimizer may skip it.

Reason 3: Stale Statistics

The query optimizer uses **table statistics** (row count, data distribution, etc.) to decide whether to use an index. If these stats are outdated:

```
-- Table had 100 rows when stats were collected → optimizer thinks full scan is fine -- Table now has 10 million rows → full scan is terrible, but optimizer doesn't know -- Fix: Update statistics ANALYZE TABLE orders; -- MySQL EXEC sp_updatestats; -- SQL Server BEGIN DBMS_STATS.GATHER_TABLE_STATS('SCHEMA', 'ORDERS'); END; -- Oracle
```

Reason 4: Index Fragmentation

Over time, as rows are inserted, updated, and deleted, the index pages become fragmented (like a hard drive). The B+ Tree leaves are no longer sequential on disk, causing random I/O.

```
-- Fix: Rebuild the index ALTER INDEX idx_name ON orders REBUILD; -- SQL Server ALTER INDEX idx_name ON orders REORGANIZE; -- Less aggressive
```

Reason 5: The Query Returns Too Many Columns

```
-- You indexed 'email', but SELECT * forces a bookmark lookup for EVERY row SELECT * FROM users WHERE email LIKE 'john%'; -- If you only need the email, the index alone is sufficient SELECT email FROM users WHERE email LIKE 'john%'; -- Covered by index, much faster
```

Reason 6: Lock Contention

Even with a perfect index, if another transaction holds a lock on the rows you need, your query waits. This looks like "slow query" but is actually a concurrency issue.

Quick Diagnostic Checklist

```
■ Run EXPLAIN/EXPLAIN ANALYZE on the query – is the index being used? ■ Check for functions on indexed columns ■ Check for implicit type conversions ■ Check selectivity – does the index filter enough rows? ■ Update table statistics ■ Check index fragmentation ■ Check for lock waits (pg_stat_activity / v$session) ■ Is SELECT * pulling unnecessary columns?
```


This is one of the hardest problems in distributed systems. Let's build up to it.

The Three Delivery Guarantees

At-Most-Once: Fire and forget. Messages might be lost, but never duplicated.

```
Producer sends → Broker may/may not receive → No retry Risk: Message lost if broker crashes
```

At-Least-Once: Retry until acknowledged. Messages are never lost, but might be duplicated.

```
Producer sends → No ACK received → Retry → Broker had actually received it Result: Message processed TWICE
```

Exactly-Once: Each message is processed exactly once. The holy grail.

Why Exactly-Once is Hard

```
Scenario: Debit ■500 from wallet 1. Consumer reads message: "Debit ■500 from user 123" 2. Consumer debits the wallet → ■500 deducted 3. Consumer tries to commit offset → CRASH! 4. Consumer restarts, reads from last committed offset 5. Reads the SAME message again → Debits ■500 AGAIN Result: User lost ■1000 instead of ■500!
```

How Kafka Achieves Exactly-Once

Kafka uses a combination of three mechanisms:

Mechanism 1: Idempotent Producer

```
Properties props = new Properties(); props.put("enable.idempotence", "true");
```

Each producer gets a unique Producer ID (PID). Each message gets a sequence number. If the broker receives a duplicate (same PID + sequence), it silently discards it.

```
Producer (PID=5): Send msg seq=1 → Broker stores it Network timeout, no ACK Retry msg seq=1 → Broker sees PID=5, seq=1 already exists → DISCARD (dedup)
```

This handles duplicates within a **single partition from a single producer**.

Mechanism 2: Transactional Producer (For Cross-Partition Atomicity)

```
Properties props = new Properties(); props.put("transactional.id", "payment-processor-1");
KafkaProducer<String, String> producer = new KafkaProducer<>(props); producer.initTransactions(); try {
    producer.beginTransaction(); // These writes are atomic – either ALL succeed or NONE
    producer.send(new ProducerRecord<>("debits", key, debitMsg)); producer.send(new ProducerRecord<>("credits", key, creditMsg));
    producer.send(new ProducerRecord<>("audit-log", key, auditMsg)); producer.commitTransaction(); } catch
(Exception e) { producer.abortTransaction(); }
```

Kafka uses a **Transaction Coordinator** and a special `__transaction_state` topic to track transaction status. Consumers configured with `isolation.level=read_committed` only see messages from committed transactions.

Mechanism 3: Consumer Offset as Part of the Transaction (Consume-Transform-Produce)

The key insight for exactly-once consume-process-produce:

```
producer.beginTransaction(); // 1. Consume messages
ConsumerRecords<String, String> records =
consumer.poll(Duration.ofMillis(100));
for (ConsumerRecord<String, String> record : records) { // 2. Process
and produce output
producer.send(new ProducerRecord<>("output-topic", processedValue)); } // 3. Commit
consumer offsets AS PART OF the transaction
producer.sendOffsetsToTransaction( offsets, // The offsets we
consumed
consumer.groupMetadata() // Our consumer group );
producer.commitTransaction(); // If this fails,
BOTH the produced messages AND the offset commit are rolled back.
// On restart, the consumer re-reads and
re-processes – but produces the same output // (idempotent), so no duplicates in the output topic.
```

The Complete Picture

```

■■■■■■■■■■ consume ■■■■■■■■■■ produce ■■■■■■■■■■ ■ Input ■ ■■■■■■■■■■→ ■
Consumer/ ■ ■■■■■■■■■■→ ■ Output ■ ■ Topic ■ ■ Producer ■ ■ Topic ■ ■■■■■■■■■■
■■■■■■■■■■ ■■■■■■■■■■ ■ commit offsets + commit produced messages IN ONE ATOMIC TRANSACTION ■
▼ ■■■■■■■■■■ ■ Transaction ■ ■ Coordinator ■ ■■■■■■■■■■
```

Summary for Interview

"Kafka achieves exactly-once through three layers: (1) Idempotent producers with sequence numbers to deduplicate retries, (2) Transactional writes that make multi-partition produces atomic, and (3) Committing consumer offsets as part of the produce transaction, so consume-process-produce is atomic. Consumers use `read_committed` isolation to only see committed messages."

5. Kafka — Partition Rebalancing

What Triggers a Rebalance?

A rebalance happens when the **mapping of partitions to consumers** needs to change:

1. **New consumer joins** the group
2. **Consumer leaves** (shutdown, crash, or missed heartbeat)
3. **New partitions added** to the topic
4. **Consumer takes too long** to process (exceeds `max.poll.interval.ms`)

What Happens During a Rebalance?

Eager Rebalancing (Default before Kafka 2.4):

```
BEFORE rebalance: Consumer A → [Part 0, Part 1] Consumer B → [Part 2, Part 3] Consumer C joins the group...
Step 1: ALL consumers STOP consuming (revoke ALL partitions) Consumer A → [] (stopped) Consumer B → []
(stopped) Consumer C → [] (waiting) Step 2: Group Coordinator reassigns partitions Consumer A → [Part 0,
Part 1] (may keep same, may change) Consumer B → [Part 2] Consumer C → [Part 3] Step 3: All consumers resume
Problem: During Step 1-2, NO messages are processed. This is the "stop-the-world" pause.
```

Cooperative/Incremental Rebalancing (Kafka 2.4+):

```
BEFORE: Consumer A → [Part 0, Part 1] Consumer B → [Part 2, Part 3] Consumer C joins... Step 1: Only REVOKE partitions that need to move Consumer A → [Part 0, Part 1] (keeps consuming!) Consumer B → [Part 2] (revokes Part 3, keeps consuming Part 2) Consumer C → [] Step 2: Assign revoked partitions Consumer C → [Part 3] (starts consuming) Key benefit: Consumers A and B never fully stop. Only the moved partition (Part 3) experiences a brief pause.
```

Problems Caused by Rebalancing

1. Duplicate Processing

```
Consumer A is processing message at offset 100. Rebalance happens before A can commit offset. Partition reassigned to Consumer B. Consumer B starts from last committed offset (95). Messages 95-100 are processed AGAIN.
```

2. Increased Latency

During rebalance, affected partitions are not consumed. For Paytm's payment processing, even a few seconds of lag can be critical.

How to Minimize Rebalance Impact

1. Tune heartbeat and session timeout:

```
# Send heartbeats frequently heartbeat.interval.ms=3000 # Don't declare consumer dead too quickly
session.timeout.ms=30000 # Allow enough time for processing max.poll.interval.ms=300000 # Don't fetch too many records at once max.poll.records=100
```

2. Use Cooperative Rebalancing:

```
partition.assignment.strategy=org.apache.kafka.clients.consumer.CooperativeStickyAssignor
```

3. Use Static Group Membership:

```
group.instance.id=consumer-host-1
```

With static membership, if a consumer restarts within `session.timeout.ms`, it gets the same partitions back without triggering a rebalance.

4. Implement idempotent processing to handle duplicates caused by rebalances.

Real World Impact at Paytm Scale

```
Scenario: Payment notification topic, 100 partitions, 20 consumers Without tuning: - One consumer pod restarts during deployment - Eager rebalance: ALL 20 consumers stop for ~30 seconds - 30 seconds of payment notifications delayed - Users don't get transaction SMS for 30+ seconds With tuning: - Cooperative rebalancing: only 5 partitions (from the dead consumer) are redistributed - Other 15 consumers continue uninterrupted - Static membership: if pod restarts within 30s, no rebalance at all - Impact: ~5 seconds of delay on only 5% of partitions
```

Quick Revision Table

Topic	Key Point
Clustered Index	Physical ordering of data, only 1 per table, leaf = actual data
Non-Clustered Index	Separate structure, many per table, leaf = pointer to data
Slow query with index	Functions on columns, low selectivity, stale stats, fragmentation
Consumer Groups	Set of consumers sharing work, 1 partition → 1 consumer per group
Exactly-Once	Idempotent producer + Transactional writes + Offset commit in transaction
Rebalancing	Triggered by join/leave, use Cooperative Sticky + Static Membership to minimize impact
Covering Index	Index includes all needed columns → no bookmark lookup needed